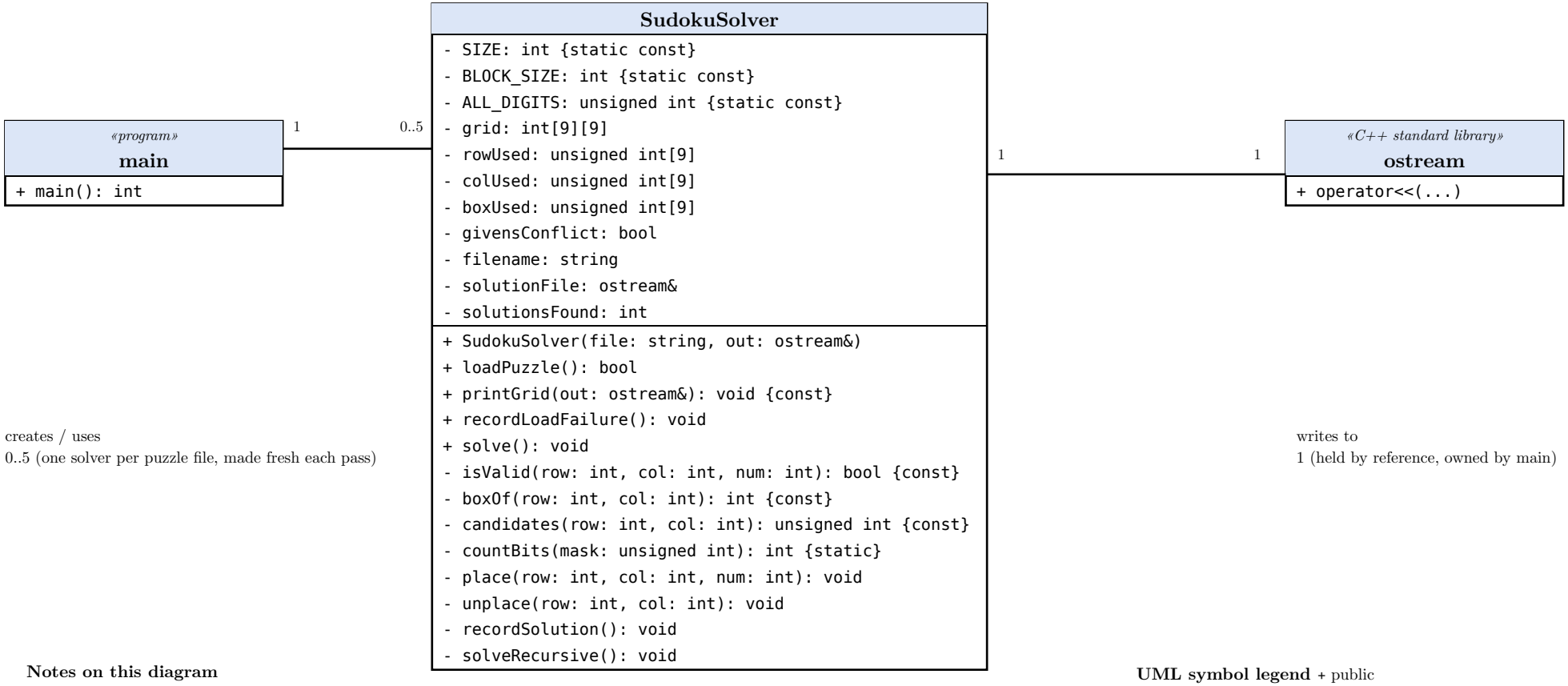
Assignment 6 – UML Class Diagram

Sudoku Solver (Assignment6.cpp) – Lucas Frias



creates / uses
0..5 (one solver per puzzle file, made fresh each pass)

writes to
1 (held by reference, owned by main)

Notes on this diagram

- This program has exactly one application class, `SudokuSolver`; it is the only box the assignment's "for each class" requirement applies to. `main()` and `ostream` are shown only to make `SudokuSolver`'s two real relationships explicit, as a UML class diagram requires.
- `SIZE`, `BLOCK_SIZE` and `ALL_DIGITS` are `static const`, so they belong to the class itself rather than to any one puzzle instance; every other attribute is a normal per-object (instance) member.
- `solutionFile` is a **reference** (`std::ostream&`), not an owned object: `main()` opens exactly one `std::ofstream` for the whole run and every `SudokuSolver` it constructs is handed that same stream, which is the one real association this class has to another type.
- There is no inheritance, composition, or second application class in this program – the `1` / `0..5` multiplicities above are the only relationships that exist to show.

UML symbol legend `+` public
`-` private
`{static}` class-level (not per-instance)
`{const}` method does not modify the object
`A --> B` association, read A "depends on / uses" B
`1`, `0..5` multiplicity at each association end